Simulation of radioactive decay: stiffness in ODE

Tomasz Chwiej

31st July 2025

1 Introduction

1.1 The law of radioactive decay

The law of radioactive decay states that the rate of decay depends linearly on characteristic constant (half-life) λ and temporary amount $N(t)$ (atomic nuclei or mass) of unstable element $dN(t)/dt = -\lambda \cdot N(t)$. Many heavy radiative isotopes decays, i.e. emit α or β particles and transform into another isotope which in turn can also decay increasing amount of radiative descendants which are known as radiative families. For three- element family (the last one, the fourth, is stable) the rates of decay forms system of linear ordinary differential equations (SODE)

$$\begin{cases} \dfrac{dN_0}{dt} = -\lambda_0 \cdot N_0 \\ \dfrac{dN_1}{dt} = \lambda_0 \cdot N_0 - \lambda_1 \cdot N_1 \\ \dfrac{dN_2}{dt} = \lambda_1 \cdot N_1 - \lambda_2 \cdot N_2 \end{cases} \implies \frac{d\vec{N}}{dt} = \vec{f}(t, \vec{N}) \implies \vec{f}(t, \vec{N}) = \begin{cases} f_0 = -\lambda_0 \cdot N_0 \\ f_1 = \lambda_0 \cdot N_0 - \lambda_1 \cdot N_1 \\ f_2 = \lambda_1 \cdot N_1 - \lambda_2 \cdot N_2 \end{cases} \tag{1}$$

This SODE has an exact solution in the following form

$$N_0(t) = N_0(0)e^{-\lambda_0 t} \tag{2}$$

$$N_1(t) = \frac{-\lambda_0 N_0(0)}{\lambda_0 - \lambda_1} e^{-\lambda_0 t} + \frac{\lambda_0[N_0(0) + N_1(0)] - \lambda_1 N_1(0)}{\lambda_0 - \lambda_1} e^{-\lambda_1 t} \tag{3}$$

$$\begin{aligned} N_2(t) = {} & \frac{-\lambda_0\lambda_1 N_0(0)}{(\lambda_0 - \lambda_1)(\lambda_2 - \lambda_0)} e^{-\lambda_0 t} + \frac{\lambda_1\left[\lambda_0(N_0(0) + N_1(0)) - \lambda_1 N_1(0)\right]}{(\lambda_0 - \lambda_1)(\lambda_2 - \lambda_1)} e^{-\lambda_1 t} \\ & + \frac{\lambda_0\lambda_1[N_0(0) + N_1(0) + N_2(0)] - \lambda_0\lambda_2 N_2(0) - \lambda_1\lambda_2[N_1(0) + N_2(0)] + \lambda_2^2 N_2(0)}{(\lambda_2 - \lambda_1)(\lambda_2 - \lambda_0)} e^{-\lambda_2 t} \end{aligned} \tag{4}$$

where $N_0(0)$, $N_1(0)$, $N_2(0)$ are the initial values.

The goal of this project is to solve SODE given in Eq.1 numerically. Because SODE is linear the r.h.s. can be defined in matrix form $d\vec{N}/dt = \boldsymbol{A}\vec{N}(t)$, diagonalization of $\boldsymbol{A}$ provides naturally the eigenvalues: λ_0, λ_1, λ_2, and if the condition $\lambda_i >> \lambda_j$ applies, for at least one pair of indices, then this SODE is stiff. We will solve the stiff SODE with implicit trapezoidal scheme applying Newton iteration and adaptive time step size.

1.2 Adaptive time step

In numerical calculations we will apply the adaptive adjustment of the time step, such approach liberates us from annoying supervising the time step - it will be changed automatically according to some imposed constraints. Below is the pseudocode for the method based on step-doubling (actually it is a conversion of an example code taken from the lecture devoted to stifness)

initialization: t_{max}, $\vec{N} = \vec{N}(0)$, $p = 2$ (trapezoidal),
$S = 0.9$, $TOL = 10^{-6}$
$\Delta t = 10^{-2}$ (better to start with low value)
$t = 0$
while $t < t_{max}$

adjust the time step:
do
$\vec{N}^{(1)} \leftarrow trapezoidal(\Delta t, \vec{N})$ (single step)
$\vec{N}^{(2)}_{\frac{1}{2}} \leftarrow trapezoidal(\Delta t/2, \vec{N})$ (first half−step)
$\vec{N}^{(2)} \leftarrow trapezoidal(\Delta t/2, \vec{N}^{(2)}_{\frac{1}{2}})$ (second half−step)
$\vec{\varepsilon} \leftarrow \frac{\vec{N}^{(2)} - \vec{N}^{(1)}}{2^p - 1}$ (errors)
$\varepsilon_{max} = max\{|\varepsilon_0|, |\varepsilon_1|, |\varepsilon_2|\}$ (maximal error)

$\Delta t_{new} \leftarrow S \cdot \Delta t \cdot \left(\frac{TOL}{\varepsilon_{max}}\right)^{\frac{1}{p+1}}$
$\Delta t_{old} \leftarrow \Delta t$
$\Delta t \leftarrow \Delta t_{new}$
while $\frac{TOL}{\varepsilon_{max}} < 1$

$\vec{N} \leftarrow \vec{N}^{(2)} + \vec{\varepsilon}$
$t = t + \Delta t_{old}$ (shift t by old time step, the new value is set for next iteration)

send output: t, $\vec{N}$
end do

This code is short and requires few operations on three-element vectors. To make it functional we need the procedure implementing the trapezoidal method.

1.3 Trapezoidal method with Newton subiteration

The trapezoidal scheme in vector notation has simple form

$$\vec{N}_{i+1} = \vec{N}_i + \frac{\Delta t}{2}\left(\vec{f}(t_i, \vec{N}_i) + \vec{f}(t_{i+1}, \vec{N}_{i+1})\right) \tag{5}$$

It is the $p = 2$ order method. The solution $\vec{N}_{i+1}$ is on the left side as well as on the right side, therefore we must derive the method which allows us to find the solution iteratively. Our experience tells that the best choice would be the Newton method.

1.3.1 Newton iteration for nonlinear vector function

The Eq.5 is implicit, we wish to know its left side but the right depends on the soultion. To solve this issue and to find the solution we apply the Newton method devised for finding the roots of multidimensional nonlinear equations. Firstly, let's rewrite Eq.5 as nonlinear function with argument (vector variable) being the vector $\vec{N}_{i+1}$

$$\vec{F}(\vec{N}_{i+1}) = \vec{N}_{i+1} - \underbrace{\vec{N}_i}_{\text{const}} - \frac{\Delta t}{2}\left[\underbrace{\vec{f}(t_i, \vec{N}_i)}_{\text{const}} + \vec{f}(t_{i+1}, \vec{N}_{i+1})\right] \tag{6}$$

this can be expressed in explicit form (just by inserting $\vec{f}$ taken from Eq.1)

$$\vec{F}(\vec{N}_{i+1}) = [F_0, F_1, F_2] \Longrightarrow \begin{cases} F_0 = N_{0,i+1} - N_{0,i} - \dfrac{\Delta t}{2}\left[(-\lambda_0 N_{0,i}) + (-\lambda_0 N_{0,i+1})\right] \\ F_1 = N_{1,i+1} - N_{1,i} - \dfrac{\Delta t}{2}\left[(\lambda_0 N_{0,i} - \lambda_1 N_{1,i}) + (\lambda_0 N_{0,i+1} - \lambda_1 N_{1,i+1})\right] \\ F_2 = N_{2,i+1} - N_{2,i} - \dfrac{\Delta t}{2}\left[(\lambda_1 N_{1,i} - \lambda_2 N_{2,i}) + (\lambda_1 N_{1,i+1} - \lambda_2 N_{2,i+1})\right] \end{cases} \tag{7}$$

In derivation of the Newton iterative formula we use Taylor series

$$\vec{F}(\vec{N}_{i+1} + \Delta\vec{N}) = \vec{F}(\vec{N}_{i+1}) + \frac{\partial \vec{F}}{\partial \vec{N}_{i+1}} \cdot \Delta\vec{N} + O\left((\Delta\vec{N})^2\right) \tag{8}$$

$$\vec{0} = \vec{F}(\vec{N}_{i+1}) + \frac{\partial \vec{F}}{\partial \vec{N}_{i+1}} \cdot \Delta\vec{N} \tag{9}$$

$$\frac{\partial \vec{F}}{\partial \vec{N}_{i+1}} \cdot \Delta\vec{N} = -\vec{F}(\vec{N}_{i+1}) \tag{10}$$

The term $\partial\vec{F}/\partial\vec{N}_{i+1}$ is the matrix of partial derivatives of $\vec{F}$ defined in Eq.7

$$\boldsymbol{G} = \frac{\partial \vec{F}}{\partial \vec{N}_{i+1}} = \begin{bmatrix} \frac{\partial F_0}{\partial N_{0,i+1}} & \frac{\partial F_0}{\partial N_{1,i+1}} & \frac{\partial F_0}{\partial N_{2,i+1}} \\ \frac{\partial F_1}{\partial N_{0,i+1}} & \frac{\partial F_1}{\partial N_{1,i+1}} & \frac{\partial F_1}{\partial N_{2,i+1}} \\ \frac{\partial F_2}{\partial N_{0,i+1}} & \frac{\partial F_2}{\partial N_{1,i+1}} & \frac{\partial F_2}{\partial N_{2,i+1}} \end{bmatrix} = \begin{bmatrix} 1 - \frac{\Delta t}{2}(-\lambda_0) & 0 & 0 \\ -\frac{\Delta t}{2}\lambda_0 & 1 - \frac{\Delta t}{2}(-\lambda_1) & 0 \\ 0 & -\frac{\Delta t}{2}\lambda_1 & 1 - \frac{\Delta t}{2}(-\lambda_2) \end{bmatrix} \tag{11}$$

By rewriting the equation (Eq.10) with matrix $\boldsymbol{G}$ we get system of linear equations

$$\Delta\vec{N} = -\boldsymbol{G}^{-1} \cdot \vec{F}(\vec{N}_{i+1}) \tag{12}$$

and now we may turn on Newton iteration scheme (upper index $k = 0, 1, 2, \ldots$ denotes the iteration)

$$\vec{N}_{i+1}^{k+1} = \vec{N}_{i+1}^{k} + \Delta\vec{N} \tag{13}$$

$$\vec{N}_{i+1}^{k+1} = \vec{N}_{i+1}^{k} - \boldsymbol{G}^{-1}\vec{F}(\vec{N}_{i+1}^{k}) \tag{14}$$

Taking the initial values from previous time step $\vec{N}_{i+1}^{0} = \vec{N}_i$ one can iteratively find sought vector $\vec{N}_{i+1}$. Because we just need to find $\Delta\vec{N}$ in each Newton iteration, this can be obtained: (1) numerically solving Eq.12 with e.g. one of LAPACK routines, or, (2) analytically as follow

$$\Delta N_0 = \frac{-F_0 g_{11} g_{22} + F_0 g_{12} g_{21} + F_1 g_{01} g_{22} - F_1 g_{02} g_{21} - F_2 g_{01} g_{12} + F_2 g_{02} g_{11}}{det(G)} \tag{15}$$

$$\Delta N_1 = \frac{F_0 g_{10} g_{22} - F_0 g_{12} g_{20} - F_1 g_{00} g_{22} + F_1 g_{02} g_{20} + F_2 g_{00} g_{12} - F_2 g_{02} g_{10}}{det(G)} \tag{16}$$

$$\Delta N_2 = \frac{-F_0 g_{10} g_{21} + F_0 g_{11} g_{20} + F_1 g_{00} g_{21} - F_1 g_{01} g_{20} - F_2 g_{00} g_{11} + F_2 g_{01} g_{10}}{det(G)} \tag{17}$$

$$det(G) = g_{00} g_{11} g_{22} - g_{00} g_{12} g_{21} - g_{01} g_{10} g_{22} + g_{01} g_{12} g_{20} + g_{02} g_{10} g_{21} - g_{02} g_{11} g_{20} \tag{18}$$

with g_{ij} being the elements of matrix $\boldsymbol{G}$ (Eq.11) while vales of F_0, F_1, F_2 are taken from Eq.7.

1.3.2 Pseudocode for Trapezoidal method with Newton iteration

Gathering information presented in previous subsection now we are in position to write the simple code for trapezoidal scheme which calculates $\vec{N}_{i+1}$ for given time step Δt

PROCEDURE INPUT: $\lambda_0, \lambda_1, \lambda_2, \Delta t, \vec{N} (\equiv \vec{N}_i)$

initialization:

$\vec{N}_{new} \leftarrow \vec{N}$ (set starting vector)

$tolerance = 10^{-10}$ (upper limit for components of $\Delta\vec{N}$ – STOP criterion)

$ITmax = 50$ (maximal number of iterations)

$it = 0$

Newton iteration:

DO

it++

$\vec{F}(\vec{N}_{new}) \leftarrow (Eq.7)$

(in $Eq.7$ replace $\vec{N}_i$ with $\vec{N}$ and $\vec{N}_{i+1}$ with $\vec{N}_{new}$)

$\Delta\vec{N} \leftarrow (Eqs. 15, 16, 17, 18)$

$\vec{N}_{new} \leftarrow \vec{N}_{new} + \Delta\vec{N}$

$\varepsilon_{max} \leftarrow max\{|\Delta N_0|, |\Delta N_1|, |\Delta N_2|\}$

WHILE($\varepsilon_{max} > tolerance$ && $it < ITmax$)

$\vec{N} \leftarrow \vec{N}_{new}$ (replace old solution with new one)

PROCEDURE OUTPUT: $\vec{N} (\equiv \vec{N}_{i+1})$

2 Practical part

1. Implement on computer the adaptive time step scheme presented in Sec.1.2 with trapezoidal scheme from Sec.1.3.2. Assume the following parameters as part of initial conditions: $\Delta t = 10^{-2}$, $N_0 = 1$, $N_1 = N_2 = 0$, $tolerance = 10^{-10}$ (in trapezoidal procedure), $t_{max} = 200$.

2. Test of correctness.
   Set parameters: $\lambda_0 = 1$, $\lambda_1 = 5$, $\lambda_2 = 50$ and $TOL = 10^{-4}$ in part of code which adjustes the time step size and solve the problem. Besides numerical outcomes calculate also the exact solution using Eqs. 2, 3 and 4. Make the plot of solution: $N_0(t)$, $N_1(t)$ and $N_2(t)$ drawing them in one figure, use logarithmic scale for time axis, add exact solutions for comparision. Make another plot showing the value of time step which is automatically set during the course of simulation, use double logarithmic scale (for time and Δt).

3. Repeat calculations for $\lambda_0 = 100$, $\lambda_1 = 1$, $\lambda_2 = 0.01$ and two tolerances $TOL = 10^{-6}$ and $TOL = 10^{-3}$. Make the same types of plots as in previous point.

4. At home prepare the report including all obtained results. Comment on the numerical results, compare the time steps set for both values of parameter TOL ($\lambda_0 = 100$, $\lambda_1 = 1$, $\lambda_2 = 0.01$), to find the striking discrepancy between them analyze the values of $N_2(t)$ in generated data files.

3 Example results

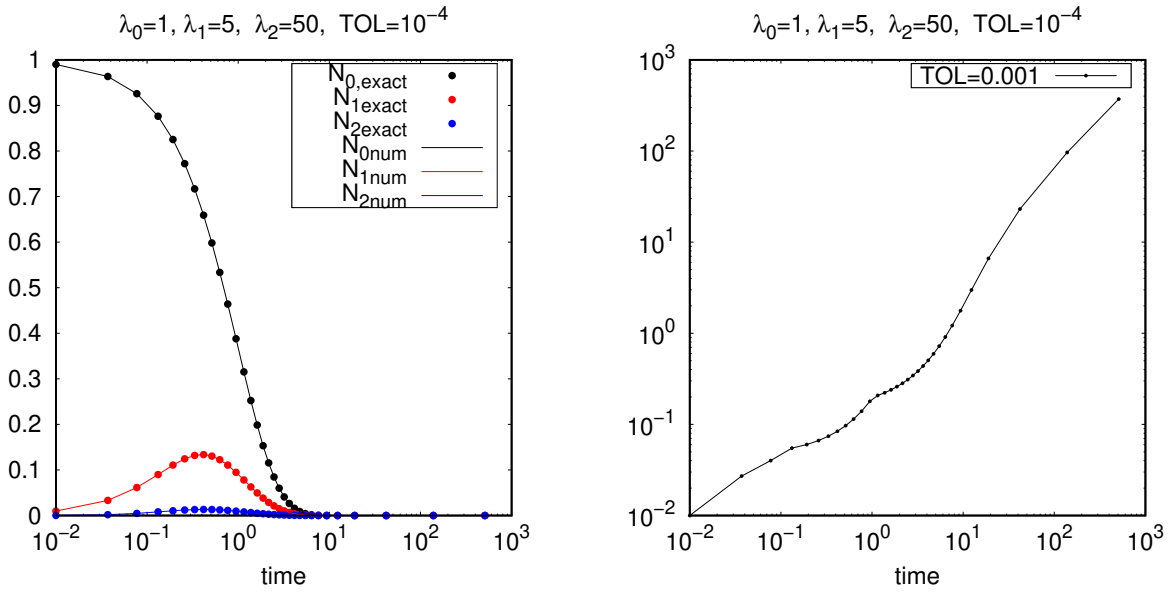

Figure 1: Solution $\vec{N}(t)$ (left) and adaptive time step Δt (right) for parameters: $\lambda_0 = 1, \lambda_2 = 5$, $\lambda_3 = 50$ and $TOL = 10^{-4}$.

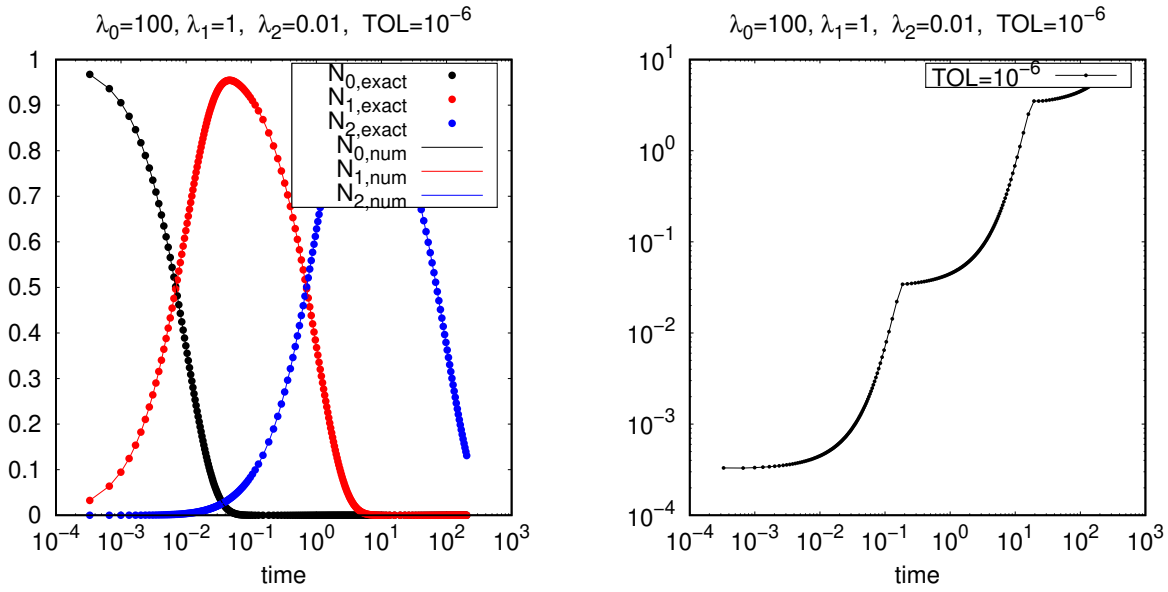

Figure 2: Solution $\vec{N}(t)$ (left) and adaptive time step Δt (right) for parameters: $\lambda_0 = 100, \lambda_2 = 1$, $\lambda_3 = 0.01$ and $TOL = 10^{-6}$.